

Measuring Disk I/O in the VM Memory-Signal Pipeline

Plan 06 — how the second capture channel is sampled and computed

1. Why a disk-I/O channel

The memory signal (APF, the fraction of guest pages that change between snapshots) cannot separate two workloads that are near-idle in memory with opposite labels: **slowburn** (a threat that writes files to disk) and **pagefault** (a benign workload that only faults pages in RAM). They alias in memory. What distinguishes them is **disk I/O** — a quantity APF cannot observe. Plan 06 adds a disk-I/O channel sampled at the same cadence as the memory snapshots, making each trajectory two-channel (APF + disk rate).

2. Data source — libvirt domblkstat

A single host-side command, issued once per snapshot, reads the guest virtual disk's I/O counters from libvirt:

```
virsh -c qemu:///system domblkstat "Kali Jeries" vda
```

It returns counters that are **cumulative since the VM booted**:

counter	meaning
rd_bytes	total bytes read from the virtual disk
wr_bytes	total bytes written to the virtual disk
rd_req / wr_req	read / write request counts (not used here)

We extract the two byte counters with awk:

```
rd=$(awk '$2=="rd_bytes"{print $3}' <<<"$stat")
wr=$(awk '$2=="wr_bytes"{print $3}' <<<"$stat")
```

3. When and where it is sampled

Inside the capture producer (capture_producer_qemu_pmemsave.sh), once per memory snapshot, right after the pmemsave dump while the **VM is paused**. Polling during the pause is cadence-safe: it adds host wall-clock but not guest-running time, so the memory-snapshot cadence is preserved. The whole block is gated by a `TIMING_DISKIO` flag — default off means the capture is byte-identical to the existing pipeline. Each sample appends one line to the cell's `diskio_trajectory.jsonl`:

```
{"seq": N, "t_emit_epoch": 1781049060.26, "rd_bytes": 502927360, "wr_bytes": 6521856}
```

`t_emit_epoch` is wall-clock at the poll (date +%s.%N). A **stride** option polls only every Nth snapshot, because the `domblkstat` call is the costly part; the cumulative counters make the per-cell rate unaffected by coarser sampling.

4. From counters to rates (the math)

Because the counters are cumulative, the per-interval rate is the difference between consecutive samples divided by the elapsed time:

```
wr_rate(i) = ( wr_bytes[i+1] - wr_bytes[i] ) / ( t_emit_epoch[i+1] - t_emit_epoch[i] )
[ bytes/s, then / 1e6 -> MB/s ]
```

Negative deltas (a counter reset on VM reboot) clamp to zero. The per-interval rates are then aggregated into per-cell features:

feature	definition
wr_rate_mean	mean write rate over the cell (MB/s)
wr_rate_max	peak write rate (MB/s)
wr_rate_p95	95th-percentile write rate (MB/s)
rd_rate_mean	mean read rate (MB/s)
wr_total	total bytes written = last - first (MB)

5. What it measures (and what it does not)

Layer: the *virtual* block device (vda) — the I/O the guest issued to its own disk, measured from the **host** via libvirt. No agent runs inside the guest.

After the guest page cache: writes buffer in guest RAM and reach vda on writeback, so short bursts lag; cumulative `wr_bytes` over a 600 s cell is accurate even though the per-interval rate is lumpy.

It is not the host's physical disk, not the page cache, and not the qcow2 image file's size.

6. Validation

The host counter was cross-checked against the guest's own view (`/proc/diskstats` sectors × 512) during a 4 GB sequential writer. They agree:

source	bytes written
host <code>virsh domblkstat vda</code>	~7.8 GB
guest <code>/proc/diskstats</code>	~7.8 GB

Host-side libvirt and the guest's own block layer report the same total, confirming the measurement is correct.

7. The pipeline (where the numbers flow)

```
producer domblkstat -> diskio_trajectory.jsonl (per snapshot, cumulative bytes)
build_cells_dir.py -> pairs each cell's diskio with its apf trajectory
diskio_features.py -> deltas / time -> per-cell wr_rate_*, rd_rate, wr_total
diskio_lift.py -> feeds the features to the classifier / masquerade test
```

8. Honest caveats

- **Page-cache lag** makes the per-interval rate lumpy; the per-cell aggregate over a long window is the robust quantity.
- **domblkstat is the costly call** (~0.5–1 s each); the stride option amortizes it for long campaigns.
- **Disk I/O is a behavioral axis, not a threat marker** — the heaviest writer measured was a benign memory-mapped workload (~140 MB/s), and a threat (memory-capped) wrote almost nothing. A second channel buys behavioral *disambiguation* of workloads that alias in the first channel, not detection.